

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 1

з дисципліни «Технології розподілених систем та паралельних
обчислень» на тему:

**«ОСНОВИ ПРОГРАМУВАННЯ ТА ЗАСОБИ СИНХРОНІЗАЦІЇ В
БАГАТОЗАДАЧНІЙ ОПЕРАЦІЙНІЙ СИСТЕМІ»**

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Старший викладач

Олександр КАЧАН

1 Мета роботи

Ознайомитися із застосуванням різних методів синхронізації при створенні багатопоточних застосунків.

2 Завдання до лабораторної роботи

2.1 Відкомпілювати й запустити запропонований приклад простий консольної багатопоточної комп'ютерної гри «Війна програмних потоків»

2.2 Розібрати текст програми ThreadWar.cpp. Знайти й підкреслити в тексті програми функції WinAPI для роботи з об'єктами синхронізації. Пояснити їх застосування.

3 Виконання лабораторної роботи

Під час виконання лабораторної роботи було виконано повноцінний рефактор вихідного коду програми. Було видалено всі не потрібні коментарі, які дублюють код, оскільки вони, згідно книги "A Philosophy of Software Design" від John Ousterhout не несуть цінності.

Також було виправлено помилки через які код не запускався та не правильно працював, винесено магічні змінні в константи, перейменовано змінні.

В процесі рефакторингу я повністю розібрався з логікою роботи програми.

Консоль в цій грі виступає в якості єдиного джерела правди, під час доступу до неї виконується блокування через mutex завдяки якому інші потоки не можуть змінити інформацію під час її читання/запису поточним потоком.

В грі кулі та вороги реалізовані через створення унікальних потоків для кожної сутності. Логіка перевірки чи було уражено противника відбувається під час його переміщення через читання інформації через консоль.

Функція яка виконує показ результату гри та розраховує чи гравець виграв була перейменована в Update, оскільки це краще відображає її суть.

Із покращень, які можна додати в цей застосунок, можна відмовитись від консолі як від головного джерела правди, а натомість зробити окрему матрицю, яка буде зберігати поточний стан програми, а після цього відображати цей стан на екран користувачу. Такий підхід дозволить зробити застосунок незалежним від GUI який ми використовуємо, а тому, наприклад, ми можемо зробити рендеринг інформації через OpenGL або через інші інструменти.

Також потрібно зауважити, що для такого простого проєкту використання потоків можливо є надлишковим, а тому якщо написати все без їх використання було б простіше, а також б це було швидше.

4 Тест розробленої програми

```
#include <windows.h>
#include <process.h>
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

HANDLE screenlockMutex;
HANDLE bulletsSemaphore;
HANDLE startEvent;
HANDLE mainThread;
CRITICAL_SECTION gameover;

HANDLE consoleIn, consoleOut;
CONSOLE_SCREEN_BUFFER_INFO consoleInfo;

long hits = 0;
long misses = 0;
long enemiesDelayFactor = 7;

const char ENEMY[] = "-\\|/";
const char BULLET = '*';
const char EMPTY = ' ';
const char GUN = '|';

const int START_DELAY = 15000;

int random(int a, int b) {
    if (a == 0 && b == 1) return rand() % 2;
    return rand() % (b - a) + a;
}

void clearConsole() {
    COORD org = { 0, 0 };
    DWORD res;
```

```

        FillConsoleOutputCharacter(consoleOut, EMPTY,
consoleInfo.dwSize.X*consoleInfo.dwSize.Y, org, &res);
    }

void paint(COORD coords, char symbol) {
    WaitForSingleObject(screenlockMutex, INFINITE);

    DWORD ignoreResult;
    WriteConsoleOutputCharacter(consoleOut, &symbol, 1, coords, &ignoreResult);

    ReleaseMutex(screenlockMutex);
}

int getSymbolAt(COORD coords) {
    char currentChar;
    WaitForSingleObject(screenlockMutex, INFINITE);

    DWORD ignoreResult;
    ReadConsoleOutputCharacter(consoleOut, &currentChar, 1, coords, &ignoreResult);

    ReleaseMutex(screenlockMutex);
    return currentChar;
}

int pressedKey(int& repeatCount) {
    INPUT_RECORD input;
    DWORD res;

    while (1) {
        ReadConsoleInput(consoleIn, &input, 1, &res);
        if (input.EventType != KEY_EVENT) continue;
        if (!input.Event.KeyEvent.bKeyDown) continue;

        repeatCount = input.Event.KeyEvent.wRepeatCount;
        return input.Event.KeyEvent.wVirtualKeyCode;
    }
}

void Update() {
    char newConsoleTitle[128];
    sprintf_s(newConsoleTitle, "Thread War! Hit: %d Miss : %d", hits, misses);
    SetConsoleTitle(newConsoleTitle);

    if (misses >= 30) {
        EnterCriticalSection(&gameover);
        SuspendThread(mainThread);

        MessageBox(NULL, "Game Over!", "Thread War", MB_OK | MB_SETFOREGROUND);
        exit(0);
    }

    if ((hits + misses) % 20 == 0) {
        InterlockedDecrement(&enemiesDelayFactor);
    }
}

void createEnemy(void* _y) {
    COORD coords;
    coords.Y = (int)_y;
    coords.X = coords.Y % 2 ? 0 : consoleInfo.dwSize.X;
    int direction = coords.X ? -1 : 1;

    while (
        (direction == 1 && coords.X != consoleInfo.dwSize.X) ||
        (direction == -1 && coords.X != 0)
    )

```

```

    ) {
        BOOL wasHitted = FALSE;

        if (getSymbolAt(coords) == BULLET) wasHitted = TRUE;
        paint(coords, ENEMY[coords.X % 4]);
        if (getSymbolAt(coords) == BULLET) wasHitted = TRUE;

        int delay = enemiesDelayFactor < 3 ? 3 : enemiesDelayFactor + 3;

        for (int i = 0; i < delay; i++) {
            Sleep(40);

            if (getSymbolAt(coords) == BULLET) {
                wasHitted = TRUE;
                break;
            }
        }

        paint(coords, EMPTY);
        if (getSymbolAt(coords) == BULLET) wasHitted = TRUE;

        if (wasHitted) {
            MessageBeep(-1);
            InterlockedIncrement(&hits);
            Update();
            _endthread();
        }

        coords.X += direction;
    }

    InterlockedIncrement(&misses);
    Update();
}

void createEnemiesThread(void*) {
    WaitForSingleObject(startEvent, START_DELAY);

    while (true) {
        if (random(0, 100) < (hits + misses) / 25 + 20) {
            _beginthread(createEnemy, 0, (void*)(random(1, 10)));
        }

        Sleep(1000);
    }
}

void createBulletThread(void* _coords) {
    COORD coordinates = *((COORD*)_coords);
    if (getSymbolAt(coordinates) == BULLET) return;

    if (WaitForSingleObject(bulletsSemaphore, 0) == WAIT_TIMEOUT) return;

    while (--coordinates.Y) {
        paint(coordinates, BULLET);
        Sleep(100);
        paint(coordinates, EMPTY);
    }

    ReleaseSemaphore(bulletsSemaphore, 1, NULL);
}

void main() {
    srand((unsigned)time(NULL));
}

```

```

DuplicateHandle(GetCurrentProcess(), GetCurrentThread(), GetCurrentProcess(),
&mainThread, 0, FALSE, DUPLICATE_SAME_ACCESS);
InitializeCriticalSection(&gameover);
startEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
screenlockMutex = CreateMutex(NULL, FALSE, NULL);
bulletsSemaphore = CreateSemaphore(NULL, 3, 3, NULL);

consoleIn = GetStdHandle(STD_INPUT_HANDLE);
consoleOut = GetStdHandle(STD_OUTPUT_HANDLE);
SetConsoleMode(consoleIn, ENABLE_WINDOW_INPUT);
GetConsoleScreenBufferInfo(consoleOut, &consoleInfo);
clearConsole();

Update();

COORD coords;
coords.X = consoleInfo.dwSize.X / 2;
coords.Y = consoleInfo.dwSize.Y - 1;

_beginthread(createEnemiesThread, 0, NULL);

while (true) {
    paint(coords, GUN);

    int keyCode, repeatCount;
    keyCode = pressedKey(repeatCount);

    /*
     * Key map:
     * Space - Fire
     * Left - Move to left
     * Right - Move to right
     */
    switch (keyCode) {
        case VK_SPACE: {
            _beginthread(createBulletThread, 0, (void*)&coords);
            Sleep(100);
            break;
        }

        case VK_LEFT: {
            SetEvent(startEvent);
            paint(coords, EMPTY);

            while (repeatCount-- > 0) {
                if (coords.X > 0) {
                    coords.X--;
                }
            }

            break;
        }

        case VK_RIGHT: {
            SetEvent(startEvent);
            paint(coords, EMPTY);

            while (repeatCount-- > 0) {
                if (coords.X < consoleInfo.dwSize.X - 1) {
                    coords.X++;
                }
            }

            break;
        }
    }
}

```

```
}  
    }  
}
```

5 Висновки

Я ознайомився із застосуванням різних методів синхронізації при створенні багатопоточних застосунків